FirstLight

Disaster intelligence for the first six hours: from raw imagery to a prioritized, routable response map.

Technical blueprint v1.0 | Geo-AI for humanitarian, civil-protection and defense-logistics response

Product in one paragraph

FirstLight ingests pre- and post-event satellite, SAR and drone imagery, detects building damage and flood extent, then fuses those outputs with road networks, population and critical facilities to answer the questions responders actually ask: *where is the need greatest, which roads still work, and which hospitals or communities are cut off?* It ships as a web map plus an API, with an LLM layer that writes grounded situation reports from the structured results, and an offline bundle for connectivity-poor field deployments.

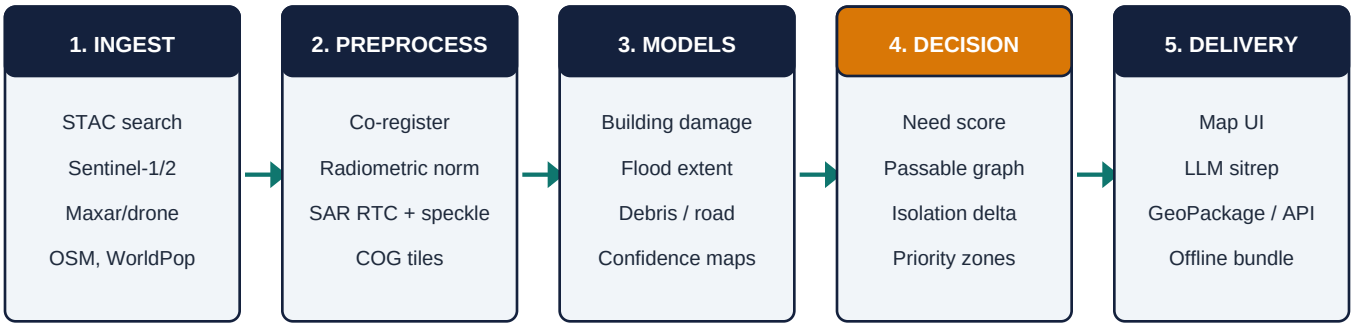
Design targets (to validate, not yet claims)

Metric	Target	How it is measured
Time to first map	< 30 min after imagery is available, for a 100 km^2 area of interest	Wall-clock on one 24 GB GPU, end to end
Building damage quality	xView2 composite score competitive with published baselines on held-out events	Leave-one-event-out evaluation
Flood extent quality	IoU >= 0.80 on held-out flood events (SAR)	Sen1Floods11 hand-labeled split plus pilot labels
Decision usefulness	Responders agree with the top-10 priority zones in a blind review	Pilot feedback sessions

What makes it different

- **Decision layer, not just polygons.** Free or manual services already produce damage and flood maps. The wedge is fusing them into ranked zones, isolation analysis and routes, in minutes.
- **Operational realism.** Confidence maps, a human-review queue for uncertain tiles, standard exports (GeoPackage, GeoJSON, COG) and an offline mode.
- **Grounded LLM.** The language model never invents numbers; it calls typed tools over the results database and cites them.

1. System architecture



Orchestrated by Prefect/Dagster. Every stage writes versioned artifacts to object storage and metadata to PostGIS.

Layer	Technology	Notes
Imagery access	pystac-client, Copernicus Data Space, Microsoft Planetary Computer / Earth Search, rasterio, rioxarray	Search by AOI and date; stream as Cloud Optimized GeoTIFF.
SAR processing	ASF HyP3 (RTC) or ESA SNAP / pyroSAR	Sentinel-1 GRD to radiometrically terrain-corrected gamma0, dB scale, speckle filter.
Training / models	PyTorch, torchgeo, segmentation-models-pytorch, TerraTorch, Lightning	Experiment tracking with Weights and Biases or MLflow.
Inference	ONNX Runtime or TensorRT, tiled with overlap blending	FP16, batch by tile; CPU fallback for offline laptops.
Spatial store	PostgreSQL + PostGIS, pgRouting, H3 indexing	Buildings, damage, zones, road graph, run metadata.
Tiles and API	FastAPI, TiTiler (rasters), pg_tileserv or PMTiles (vectors)	OGC-friendly outputs; JWT auth, per-tenant buckets.
Frontend	MapLibre GL JS, deck.gl for overlays	Before/after swipe, time slider, confidence toggle, review queue.
Orchestration	Prefect or Dagster, Docker, Kubernetes (cloud) / Docker Compose (offline)	Idempotent, resumable stages; event-triggered runs.
LLM layer	Claude API with tool use	Fixed tool set over PostGIS; no free-form SQL.

2. Data sources

Licensing matters if you plan to sell. Several research datasets are non-commercial, so use them to prototype and benchmark, then build a commercially clean training set from open-license data and pilot labels. Confirm every license before shipping.

Source	Use	Scale / notes	Commercial?
xBD (xView2)	Building damage pre/post labels, 4 damage classes	Roughly 19 events, 800k+ building polygons, ~0.5 m class imagery	Non-commercial license: benchmark and prototype only
Sen1Floods11	SAR and optical flood segmentation	4,831 chips of 512x512 px, 11 flood events, hand-labeled subset	Open (CC BY 4.0); verify
Sentinel-1 / Sentinel-2	Flood extent, change, vegetation / burn	Free via Copernicus; revisit varies by constellation status, so check current coverage	Free and open, commercial use allowed

Source	Use	Scale / notes	Commercial?
Maxar Open Data	High-resolution event imagery	Released for major disasters; coverage is event-driven	Check terms; often non-commercial
OpenStreetMap	Roads, hospitals, shelters, buildings	Quality varies by region; sync via Overpass or extracts	ODbL: attribution and share-alike
Building footprints	Skip the localization stage where available	Microsoft and Google Open Buildings datasets	ODbL / CC BY 4.0; verify
WorldPop / GHSL	Population per cell for need scoring	~100 m grids	Open licenses; verify
Pilot labels (yours)	Fine-tuning, local calibration	Built with partner analysts via a review tool	Yours, under data-sharing agreement

3. Model design

3.1 Building damage (optical, pre/post)

Two-stage design mirrors the strongest xView2 approaches. Stage A localizes buildings (skip it when footprints exist). Stage B classifies damage per pixel or per footprint into five classes: background, no damage, minor, major, destroyed. Start with early fusion (stack pre and post as six channels) as a quick baseline, then move to a Siamese encoder with feature differencing at each scale.

```
import torch, segmentation_models_pytorch as smp

# Baseline: early fusion of pre/post RGB (6 channels), 5 classes
model = smp.Unet(encoder_name="resnet50", encoder_weights="imagenet",
                 in_channels=6, classes=5)

def loss_fn(logits, target, w):          # w: class weights, rarer classes weighted up
    ce = torch.nn.functional.cross_entropy(logits, target, weight=w)
    dice = smp.losses.DiceLoss(mode="multiclass")(logits, target)
    return ce + dice
```

3.2 Flood extent (SAR)

Ship a physics-based baseline first: Sentinel-1 VV/VH in dB, compare event image to a dry reference, threshold water (Otsu or a fixed backscatter cut) and mask permanent water. Then train a U-Net or fine-tune a geospatial foundation model on Sen1Floods11 plus your own labels. SAR works through cloud, which is the key operational advantage for floods. Known failure modes to handle explicitly: urban double-bounce (flooded buildings can look bright), radar shadow in steep terrain, and wind-roughened water.

3.3 Foundation-model path

Fine-tune a pretrained geospatial model (for example NASA/IBM Prithvi-EO, or Clay) with TerraTorch for flood and burn-scar segmentation. Compare against the plain U-Net on held-out events. Adopt it only if it wins on generalization to new regions, because that is what buyers care about, not in-distribution accuracy.

3.4 Training recipe

- **Split by event, never by tile.** Tiles from one disaster are highly correlated; random splits inflate scores.
- **Augmentation:** flips, 90 degree rotations, brightness and contrast jitter, simulated haze, random pre/post misregistration of 1-3 px.
- **Class imbalance:** weighted CE plus Dice, oversample tiles containing major or destroyed buildings.
- **Compute:** a single 24-48 GB GPU with mixed precision is enough for a first model; expect hours to about a day on xBD-scale data.
- **Calibration:** temperature scaling on a validation event; export per-pixel confidence and use it to drive the review queue.

4. Decision layer

4.1 Zone need score

Aggregate to hexagons (H3 resolution 8 to 9, roughly neighborhood scale) and compute a transparent, tunable score so partners can inspect and adjust it:

```
need(z) = w1*D(z) + w2*Pexp(z) + w3*Iso(z) + w4*Crit(z)        weights sum to 1, set with partners

D(z)    = (1.0*n_destroyed + 0.6*n_major + 0.2*n_minor) / n_buildings(z)
Pexp(z) = affected_population(z) / max_over_zones(affected_population)       (WorldPop x damage)
Iso(z)  = share of zone population whose travel time to nearest functioning hospital
          exceeds T minutes on the passable graph (vs baseline graph)
Crit(z) = count of critical facilities (hospital, school, shelter, water) damaged or cut off
```

4.2 Passable-road graph and isolation analysis

- Build the road graph from OSM with osmnx or osm2pgrouting; store in PostGIS with pgRouting.
- Mark an edge blocked or penalized when it intersects the flood mask above a coverage threshold, or lies within a buffer of destroyed buildings (debris proxy). Without depth data, treat flooding as binary and expose the threshold as a user setting.
- Compute travel-time isochrones from hospitals and depots on the baseline graph and on the post-event graph. The *difference* is the isolation delta, which is the most compelling output for dispatchers.
- Provide route queries with explicit uncertainty: low-confidence edges are flagged for ground verification rather than silently trusted.

5. LLM situation-report layer

Use tool calling over a fixed set of typed functions so every number in a report traces to the database. Do not allow free-form SQL from the model.

Tool	Returns
get_event_summary(event_id)	AOI, image dates, counts by damage class, flooded area, run quality metrics
top_zones(event_id, n, weights?)	Ranked H3 zones with component scores
isolated_facilities(event_id, type)	Facilities with baseline vs post-event travel time
route(event_id, from, to)	Path, distance, time, blocked-edge flags, confidence
low_confidence_areas(event_id)	Tiles recommended for human review or drone tasking

- Prompt rules: cite the tool result for every figure, state image dates and sensor, never extrapolate beyond the data, and surface known limitations (cloud, SAR artifacts, resolution) in each report.
- Output formats: situation report in plain language, a one-page briefing PDF, and a CAP-style alert draft for human sign-off.
- Keep a human in the loop for anything operational. The LLM drafts; analysts approve.

6. Inference and serving

Large scenes must be tiled with overlap and blended to avoid seam artifacts. A 100 km^2 area at 0.5 m is about $4x10^8$ pixels per date, or roughly 380 tiles of 1024x1024 px, which is minutes of GPU time.

```
from rasterio.windows import Window

def tile_windows(h, w, size=1024, overlap=128):
    stride = size - overlap
    ys = list(range(0, max(h - overlap, 1), stride))
    xs = list(range(0, max(w - overlap, 1), stride))
    for y in ys:
        for x in xs:
            yield Window(x, y, min(size, w - x), min(size, h - y))

# Per window: read pre/post -> normalize -> pad to size -> model -> softmax
# Accumulate probabilities with a 2D Hann weight, divide by weight sum,
# argmax at the end, then polygonize with rasterio.features.shapes.
```

Stage (100 km2 AOI)	Target
Fetch, co-register, normalize	~5 min
Tiled inference, FP16, one GPU	5-10 min
Vectorize and attach to footprints	~3 min
Scoring, routing, isolation delta	~2 min
Report generation	< 1 min

7. Evaluation plan

- **Damage:** xView2 composite (0.3 x localization F1 + 0.7 x damage F1), plus per-class F1. Report major and destroyed separately; they matter most.
- **Flood:** IoU, precision, recall on held-out events; separate urban and rural slices.
- **Generalization:** leave-one-event-out and leave-one-region-out. Publish the gap between in-distribution and held-out scores honestly.
- **Calibration:** expected calibration error and reliability diagrams; confidence must predict where errors occur.
- **Decision quality:** blind comparison of top-N priority zones against analyst-produced rankings; time-to-map versus the manual baseline.
- **Regression suite:** a frozen set of past events re-run on every model change, with metrics stored per run.

8. Repository structure

```
firstlight/
  data/            # DVC-tracked; raw/ interim/ labels/ (never committed to git)
  ingest/          # stac_search.py, sar_rtc.py, osm_extract.py, worldpop.py
  models/
    damage/        # dataset.py, model.py, train.py, export_onnx.py
    flood/         # sar_threshold.py, unet.py, terratorch_cfg.yaml
  decision/        # h3_zones.py, need_score.py, graph.py, isolation.py
  llm/             # tools.py, prompts/, report_templates/
  api/             # FastAPI app, auth, schemas
  web/             # MapLibre frontend
  infra/           # docker/, k8s/, offline_bundle/
  eval/            # metrics.py, event_splits.yaml, regression_runs/
  docs/            # model cards, data cards, pilot playbook
```

9. Eight-week roadmap

Week	Build	Exit criterion
1	Repo, DVC, STAC ingest, xBD and Sen1Floods11 loaders, tiling utilities	Reproducible data pipeline on two events
2	SAR flood baseline (threshold) and early-fusion damage U-Net	Baseline metrics logged to tracker
3	Siamese damage model, flood U-Net, class-weighted training	Leave-one-event-out report
4	H3 zoning, need score, OSM graph in PostGIS, isolation delta	Ranked zones plus isolation map on a past event
5	FastAPI, TiTiler, MapLibre UI with swipe and time slider	Click-through demo on a laptop
6	LLM tools and sitrep generation, confidence layer, review queue	Auto-report where every number traces to a tool call
7	ONNX export, offline bundle, performance tuning, regression suite	Meets the time-to-map target
8	Pilot packaging: model card, data card, demo video, partner onboarding	Pilot agreement in discussion

10. Go-to-market

Buyers and channels

- **Primary:** national and regional civil-protection agencies, humanitarian clusters, UN agencies and international NGOs (rapid assessment and logistics teams).
- **Secondary:** insurers and reinsurers (event footprint and claims triage), infrastructure operators, and defense or military engineering units for humanitarian assistance and disaster relief logistics.
- **Channels:** pilot programs, accelerator and innovation programs focused on climate and defense, humanitarian innovation funds, and integrator partnerships with GIS vendors.

Pricing hypotheses to test

Model	Description
Pilot	Free or low-cost for one real or historical event in exchange for feedback, labels and a case study
Annual license	Per-agency subscription with an included number of event activations
Per-activation	Pay per event and area for insurers and one-off users
Offline appliance	Premium license for air-gapped or low-connectivity deployments

Moat

- Locally labeled data and feedback loops from pilots, which improve regional accuracy over time.
- Workflow integrations (QGIS, ArcGIS export, CAP alerts, field-data tools) that make the product sticky.
- Trust: transparent model cards, calibrated confidence and honest published benchmarks.

Competitive landscape to study

Copernicus Emergency Management Service and UNOSAT (trusted, often manual or semi-automated, slower), plus commercial players in flood and damage analytics and satellite operators with their own analytics. Position FirstLight as faster, integrated with routing and logistics, and deployable offline.

11. Risks, licensing and ethics

Risk	Mitigation
Non-commercial dataset terms	Prototype on xBD, then retrain on commercially clean data and pilot labels; get legal review before launch
Domain shift across regions and building types	Leave-region-out testing, local fine-tuning, confidence-driven review
Cloud cover on optical imagery	SAR-first for floods; flag optical gaps; support drone uploads
Image availability and latency	Pre-negotiated tasking and archive access; degrade gracefully to free Sentinel data
Over-trust in automated output	Calibrated uncertainty, mandatory human sign-off, clear limitations in every report
Dual-use and export controls	Scope to humanitarian, logistics and damage assessment; no targeting features; review export rules before selling to defense customers
Privacy and safety	Aggregate sensitive outputs where appropriate; access control; data-sharing agreements for pilot data

Suggested next step: build weeks 1-3 on one flood event and one earthquake or storm event from xBD, and record a 90-second demo. A working before/after swipe with ranked zones and isolation analysis is the strongest asset for pilots and funding conversations.